TEC | Tecnológico de Costa Rica

II Semestre, 2026

Taller de Programación

Proyecto #1 – Sistema de Gestión de Transporte Público

Profesor: Ing. Jean Carlos Miranda Fajardo

Estudiante: Kristian Araya Valverde

Carné: 2026094656

Índice

Manual de Usuario - - - - - Pág. 3 - 4

Menú de opciones administrativas - - - - - Pág. 4 - 10

Descripción del problema - - - - - Pág. 11

Diseño del programa - - - - - Pág. 12 – 13 (INCOMPLETO)

Librerías Usadas - - - - - Pág. 14

Análisis de resultados - - - - - Pág. 14

Bitácora de trabajo - - - - - Pág. 15 – 19

Conclusiones - - - - - Pág. 20

Manual de Usuario

Instrucciones de compilación, ejecución y uso

Antes de siquiera pensar en utilizar el programa, debemos asegurarnos de tener descargada la versión más reciente del IDLE de Python, una vez la tenemos, vamos al repositorio de GitHub (enviado por la persona estudiante) y descargamos todo el proyecto.

Una vez descargado veremos que tenemos 2 carpetas: La documentación (en la que se incluye este archivo que usted está leyendo) y el Programa, el cual trae el archivo **proyecto_1_transporte_público.py** y todos los archivos.txt que este necesita par su necesaria y efectiva ejecución.

PASOS:

1 – Abra el IDLE de Python y en la esquina superior izquierda identifique el boton de **File**, una vez ahí haga clic en **Open** y busque la carpeta donde está el archivo.py del proyecto.

2 – Una ve este esté abierto, podrá ver el código y arriba en la barra una opción llamada **Run**, en esta haga clic en **Run Module (en caso de pedirlo, guarde el archivo)**

3- Una vez en ejecución se le presentara un menú como este en la consola IDLE:

Bienvenidos al Sistema TP

=== MENÚ PRINCIPAL ===

(1) Opciones Administrativas

(2) Opciones de Usuario

(3) Salir

Elija una opcion

4 – En la parte de abajo debe de ingresar el número de la opción que desea ejecutar, en caso de ingresar un número no válido se le va a informar, las opciones tienen las siguientes funciones:

(1) Opciones Administrativas: Da ingreso al menú de administración de la empresa transportadora, (**IMPORTANTE:** Para ingresar a este menú debe de tener una clave y una contraseña autorizadas, las cuales se sitúan en el archivo "login.txt") más detalles sobre este menú más adelante.

(2) Opciones de Usuario: Da ingreso al menú para usuarios de la plataforma, este es el menú común, pues no se necesita de ninguna clave o usuario para ingresar a este. (**AVISO:** Hasta este momento, este menú solo existe en representación gráfica, ninguna de sus opciones tiene una utilidad)

(3) Salir: Termina el programa, no cierra el IDLE, simplemente termina la función **menu_principal()**, la cuál es la que se inicia apenas corres el código.

Menú de Opciones Administrativas

Cuando ingresemos por primera vez a este menú, nos vamos a topar con el siguiente display:

Menú de administradores, porfavor ingrese sus credenciales

Usuario:

Contraseña:

En esta parte, usted deberá de ingresar sus credenciales para poder acceder al menú de administración, esto porque desde este menú se pueden hacer operaciones importantes como el manejo de modelos, unidades, fechas de salida e incluso credenciales de conductores, por lo que un usuario normal no debe de ser capaz de acceder aquí. En caso de que se ingrese mal algún parámetro, no se le dira al usuario que ingreso mal, esto por razones de seguridad, solo se le da un mensaje genérico (Usuario o contraseña incorrectos, vuelva a intentarlo), en caso de que sean correctas, se abre paso al menú de administración el cual se ve así:

¡Bienvenido!

=== MENÚ ADMINISTRATIVO ===

(11) Gestión de modelos de autobús

(12) Gestión de unidades

(13) Gestión de conductores

(14) Gestión de rutas

(15) Gestión de paradas por ruta

(16) Programación de salidas

(17) Consultar historial de abordajes

(18) Regresar al menú principal

Elija una opción:

Aquí en elegir una opción, podemos elegir una de las que ahí se muestran, de las cuales vamos a describir que puede hacer cada una de ellas:

(11) Gestión de modelos de autobús: Una vez entremos a este modo se nos presentan las opciones:

=== GESTIÓN DE MODELOS DE AUTOBÚS ===

(1) Incluir modelo

(2) Eliminar modelo

(3) Modificar modelo

(4) Mostrar modelos

(5) Volver

Elija una opción:

(1) Incluir modelo: Esta opción nos permite agregar un nuevo modelo de autobús a el archivo "modelos.txt", para agregar un nuevo modelo, el programa le pedirá los siguientes datos: Un modelo, una marca, cantidad de asientos y cantidad de espacios a pie. (*Ninguna de estas tiene validaciones, así que es importante saber que, si usted ingresa por ejemplo "Mercedes" en cantidad de asientos, se lo va a aceptar*)

(2) Eliminar modelo: Esta opción del programa le permite al usuario eliminar uno de los modelos antes registrados, para poder eliminar un modelo, se debe de cumplir con las siguientes condiciones:

A. El modelo a eliminar tiene que estar registrado en el archivo de "modelos.txt" con antelación.
B. No se puede eliminar un modelo que ya este asociado a una unidad

Si cumple con todo lo mencionado, le dejara eliminar el modelo con solo insertar su descripción en el campo solicitado.

(3) Modificar modelo: Esta opción del programa le permite al usuario el modificar uno de los modelos antes registrados, para poder modificar un modelo, se debe de cumplir con que el modelo exista en el archivo de "modelos.txt".

(4) Mostar modelos: Esta opción es simple, intenta mostrar de forma cómoda los modelos con sus datos al usuario para que este no tenga que tener abierto el archivo.txt por aparte.

(12) Gestión de unidades: Una vez entremos a este modo se nos presentan las opciones:

=== GESTIÓN DE UNIDADES ===

(1) Incluir unidad

(2) Eliminar unidad

(3) Modificar unidad

(4) Mostrar unidades

(5) Volver

Elija una opción:

(1) Incluir unidad: Al seleccionar esta opción nos va a permitir agregar una nueva unidad a el archivo de "unidades.txt", ojo pues esta la condición de que en esta función debemos escoger un modelo para asociar a la unidad (*no he probado que pasa si "modelos.txt" esta vacío y se intenta usar esta función),* luego de ahí, se te piden las cosas normalmente, una placa, el año de la unidad y empresa que la maneja.
(2) Eliminar unidad: Esta función le permitirá al usuario eliminar una de las unidades seleccionadas al ingresar la placa de la unidad, la única condición para eliminar una unidad es que esta exista en "unidades.txt".
(3) Modificar unidad: Nos permite cambiar los datos de una unidad que ya exista en el txt, debemos de volver a escoger un modelo e ingresar los datos.
(4) Mostar unidades: Intenta enseñar de forma cómoda al usuario una interfaz con todas las unidades en "unidades.txt".

(13) Gestión de conductores: Una vez entremos a este modo se nos presentan las opciones:

=== GESTIÓN DE CONDUCTORES ===

(1) Incluir un conductor

(2) Eliminar un conductor

(3) Modificar información de un conductor

(4) Mostrar a los conductores

(5) Volver

Elija una opción:

(1) Incluir conductor: Al seleccionar esta función nos va a permitir agregar un conductor a el archivo de "conductores.txt", para poder agregar un conductor debemos de asegurarnos que este no tenga una cédula repetida o nos dará error, los datos del conductor son los siguientes: cédula, nombre completo, tipo de licencia, vencimiento de la licencia y las horas de jornada máximas por día de ese conductor.
(2) Eliminar conductor: Nos permite ingresar la cédula de un conductor para eliminarlo de nuestro archivo de "conductores.txt", la condición para realizar esta eliminación es que este ya exista en el archivo antes de hacer la operación.
(3) Modificar info del conductor: En caso de ser necesario, podemos modificar la información de un conductor con esta función, pero solo de los que ya existen en el archivo.
(4) Mostrar conductores: Tira una pequeña lista gráfica de todos los conductores que se han registrado en el archivo.

Para todos los casos anteriores "(5) Volver" vuelve del menú en el que se encuentran al menú administrativo. Las funciones de la 14 a la 17 se identificarán como "números no válidos", esto se debe a que no se desarrollaron.

De igual manera pasara al intentar acceder a las opciones de usuario, si bien el menú existe, ingresar los números que ahí se encuentran no tendrá ningún efecto sobre el programa y retornara que no son válidos en cuanto a una selección

Existen 2 funciones auxiliares que se llaman dentro de las funciones principales, el usuario interactúa con ellas de forma indirecta, estas son:

detecta_repetidos: Aunque se llama así, en realidad puede cumplir con 2 funcionalidades dentro del programa. La primera es, como su nombre lo indica, detectar si en el archivo que estamos leyendo, existen ya sea un modelo, una unidad o una cédula que este repetida y le avisa al usuario que no se permiten ingresar datos repetidos (Para mal, esta función válida solo exactamente el dato que ya existe, por ejemplo, si yo ingreso un modelo llamado Labrador, esta función no logra evitar que otro modelo se llame LABRADOR). La segunda utilidad de esta función es que esta se puede usar para determina si el modelo que estamos buscando eliminar, existe o no es el archivo txt que se le envía.

La lógica de esto es que detecta repetidos pregunta: En esta línea, ¿esta descripción x (donde con descripción nos referimos a un dato, los datos van separados por ;) es igual a este otro dato nuevo que el usuario quiere ingresar? Si es True, entonces evita que se repita, si es False, pasa.

Para su segunda funcionalidad pregunta, lo mismo, ¿qué pasa? Si el modelo nuevo es igual al que ya está registrado, entonces la función sabe que existe dentro del archivo y lo eliminara, básicamente preguntamos si se repite para saber si existe y poder eliminarlo.

comprobar_asociado: Esta recibe 3 parámetros a la hora de ejecutarse: **un archivo** donde buscaremos **un dato** en **un campo especifico**. ¿Pero que son estos tres parámetros? En el caso de hasta donde llego este programa, esta función solo se usa en una ocasión, para saber si una unidad tiene una modelo ligada a ella. El archivo será “unidades.txt”, el dato será el modelo y el campo será el campo 2 de cada línea en “unidades.txt”, esto porque ahí es donde se

encuentra el modelo que decidimos ligar a la unidad cuando usamos la función de agregar unidades. Por lo tanto, eliminar modelo llama a esta función y dice: El usuario quiere eliminar X modelo, ¿este modelo X está asociado a una unidad Y? Si comprobar_asociado le dice que si, entonces es imposible eliminar ese modelo sin antes eliminar la unidad.

Descripción del Problema

Se le solicita al estudiante diseñar una aplicación que trabaje en la consola IDLE de Python que simule un programa de operaciones para una empresa de transporte público, dicho sistema tiene que tener varias funciones de 2 lados: Administración y de Usuarios.

Para las funciones administrativas se requieren una gran variedad de funciones que nos ayuden a manejar todos los posibles escenarios que ocurran en el sistema, como, por ejemplo: Tener que registrar, eliminar o modificar unidades del transporte, así como asignarles sus debidos conductores. Poder crear rutas donde operaran las unidades, con intervalos de tiempo que tengan sentido y sean fáciles de comprender. En general debemos ser capaces de manejar toda la logística de la empresa desde aquí, controlar las unidades, los conductores y las rutas, también un historial de que reservaciones se han hecho y cuánto dinero se ha recaudado hasta el momento.

Por otro lado, para nuestras funciones como usuario, debemos tener la capacidad de poder consultar cuales son las salidas programadas por la empresa, de querer ir en una de ellas, debemos de poder registrar nuestro abordaje al sistema, usando nuestro nombre y con ello elegir la cantidad de asientos que vayamos a necesitar, con esto también debemos de ser capaces de cancelar nuestra reservación en caso de que lo requiramos.

Diseño del programa

Decisiones de desarrollo, algoritmos usados

Para menús: Básicamente los menús son funciones con prints e inputs, al principio de todo mi programa empiezo por hacer siempre **menu_principal()** esto hace que se muestre el primer menú de todos y por ende marca el inicio de la aplicación, luego de ahí, los inputs son controlados por números, donde ejemplo, el programa te pide "Escoja una de las opciones: ", entonces vez el IDLE elegís un número y al ingresarlo llama a otra función de menú o a una función operativa.

La lógica ideada va algo así:

menu_principal() inicia todo

despliego prints de menu principal

pido input al usuario

depende del número muestro un nuevo menú

(caso especial del menú admin donde primer le pido al usuario sus claves)

Para funciones de "Incluir": La mayoría de nuestras funciones de incluir trabajan primero pidiendo los datos al usuario y luego registrando la información con un append al final (excepto incluir unidades, quien debe de abrir primero en modo read, modelos.txt para añadirse un modelo a la unidad)

La lógica ideada va algo así:

Hago un print pidiendo los datos al usuario

dato_UNICO asígnele el valor que el usuario vaya a agregar

llame a la función detecta_repetidos para saber si dato_UNICO ya existe

mientras me diga que dato_UNICO ya existe

pídale al usuario que ingrese un dato_UNICO diferente

una vez se ingrese uno diferente

pida el resto de datos

meta todos esos datos en una variable de un solo string ordenado, una línea

abra el archivo designado en modo append para añadirlo al final

escriba en el archivo la variable de la línea completa y agréguele un salto de línea al final

Librerías Usada

No se llego a usar ninguna librería para este proyecto

Análisis de Resultados

Objetivos alcanzados y no alcanzados

Menú principal y navegación entre menús - Completa
Los menús que existen son accesibles y funcionales, se puede navegar entre ellos, los que no sirven se debe a que no se desarrollo esa parte del programa.

Control de acceso administrativo - Completa
Las contraseñas y usuarios logran cumplir su función de evitar el acceso de forma forzada al sistema.

Gestión de modelos de autobús y de unidades - Parcial
Se completaron todas sus funciones, sin embargo, a pesar de que funcionan como deberían, las validaciones son prácticamente inexistentes, creo que las únicas validaciones son que existan dentro de un archivo y que no se repitan en el caso de las funciones de eliminar y modificar. Para incluir modelos, se válida que si esta enlazado un modelo con una unidad, no se pueda borrar

Gestión de conductores – Parcial
Pasa igual que el caso anterior, las validaciones son muy poco máximo 2, por lo que, si se ingresara, ejemplo (Cédula: ihf9h34y) el sistema lo deja pasar y lo registra en el archivo.

NO SE REALIZO NINGUNA DE ESTÁS:
Gestión de rutas y paradas por ruta
Programación de salidas con validación de traslape y jornada
Registro y anulación de abordajes
Consultas con filtros (salidas y abordajes)

Persistencia de la información en disco – Completa
Todo se almacena en archivos .txt respectivos que logran permanecer cuando se cierra el programa y este se vuelve abrir:

Bitácora de Trabajo

Registro del trabajo realizado en días específicos

Fecha de realización	Horas dedicadas	Descripción de Actividad
19/9/2026	6:30pm – 7:28 pm (58 minutos) Leyendo de nuevo la práctica, para usar git y GitHub, arreglando errores con las carpetas 7:30pm – 8:20pm (50 minutos) Trabaje en el menú principal y plantee la idea de cómo manejar el sistema de usuarios y contraseñas	Creación de carpetas Repositorio de GitHub Conexión con el repositorio Local Documentación Externa (Un Word editable por el momento)
22/9/2026	6:10pm – 7:02pm (52 minutos) Avance del sistema de contraseña (concepto) 7:03pm – 7:15pm (12 minutos) Intentar crearlo ya con el archivo	Avanzar con el sistema de inicio de sesión (admin) Se intento lograr que el archivo se maneje con listas, por ahora lo dejo incompleto y en un archivo separado.
23/9/2026	4:10pm – 5.13pm (1 hora, 3 minutos)	Se terminan de crear los tres menús principales (Menu principal, admin, usuario) Los menús se trataron de crear de varios modos, pero al final me decidí hacerlos con un while y condicionarlos con True y False
28/9/2026	3:00pm – 3:24pm (24 minutos)	Se terminan las contraseñas de manera que se conecten con los menús de manera satisfactoria

29/9/2026	3:20pm – 3:25pm (5 minutos) Arreglar confusión	Se arregla la confusión de ponerle el login al sistema de usuarios en vez de al de admin.
	3:25 pm – 4:20pm (55 minutos) Trabajo de lógica	Trabajar en la lógica para las funciones de gestión de modelos
30/9/2026	10:00am – 10:40am (40 minutos) Trabajo lógica y creación de función auxiliar	Se crea una función que nos ayudara a detectar repeticiones en un archivo txt deseado detectar_repetidos()
	10:50am – 11:47am (57 minutos) Conectar auxiliar con la función principal	Se crea la función de agregar modelos, por ahora solo pide los inputs, pero valida que los modelos no se repitan.
	11:50am – 12:05pm (15 minutos) Se agregan las funciones de inclusión de modelos al archivo principal	Se agregan las nuevas funciones de detección de repetidos y la adición de modelos a el archivo del programa principal
1/10/2026	5:50pm – 6:06pm (16 minutos) Agregar modelos se termina	Agregar modelos se termina y ya funciona correctamente, solo hacen falta sus validaciones
	6:07pm – 6:32pm (25 minutos) Se trabaja la función de eliminar modelos()	Se agrega la función de eliminar modelos a media, se guarda lo que queremos, pero aún no borramos lo que no queremos.

1/10/2026	9:00pm – 9:37pm (37 minutos) Finalización de la función eliminar modelos	Se termino la función de eliminar modelos y se corrigieron sus errores. Hace falta de sus validaciones respectivas
2/10/2026	9:08am – 9:35am (27 minutos)	Se crea la función de modificar un modelo, también se intentó darle una vista más ordenada al código.
	10:00am – 10:40am (40 minutos)	Se termina la función de mostrar los modelos y se conectan todas al menú de manera que cada una sirva y puedan trabajar juntas. (FALTA VALIDAR CASOS)
	3:00pm – 4:40pm (1 hora, 40 minutos)	Editar el documento, trabajar en el un poco, también inicio creando, incluir unidades, la cual dio problemas debido a que debe de llamar 2 archivos diferentes.

2/10/2026	7:25pm – 8:34pm (1 hora, 9 minutos)	Se crea las funciones para eliminar unidades, mostrar unidades y modificar unidades, se conectan al menú principal y ya funcionan entre ellas, sin sus validaciones por el momento
	9:00pm – 10:07pm (1 hora, 7 minutos)	Se crean todas las funciones de la gestión de conductores, incluir, eliminar, modificar y mostrar conductores. Se enlazan al menú principal para que sea posible usarlas de manera eficaz. No tienen sus validaciones completas.
3/10/2026	2:33pm – 3:06pm (36 minutos)	Trabajo en los menús, mejorando y eliminando líneas innecesarias y que estaban causando errores, como llamadas recursivas (No subo git, no he terminado de modificar, pero debía hacer unas revisiones para un proyecto de fundamentos)
	6:35pm – 6:45pm (10 minutos)	Se terminan los cambios sobre el menú que se estaban realizando con anterioridad. (Ahora si se sube el commit)

3/10/2026	6:58pm – 7:46pm (48 minutos)	Se crea la función auxiliar de comprobar_asociado, la cual nos va a ayudar a poder comprobar de manera general si dos archivos tienen elementos asociados (Todavía se esta probando)
	8:12pm – 9:18ppm (1 hora, 6 minutos)	Se agrego la función comprobar asociado, solo a eliminar modelos, esto pues unidades y conductores solo la utilizan cuando requieren de una salida, y las salidas no se crearon. Se trabaja en el documento de documentación externa.
	9:27pm – 11.40 (2 horas, 13 minutos)	Se realizan los últimos cambios posibles a la documentación externa, queda incompleta (no se tomará en cuenta para evaluación)

Conclusiones

Punto de vista del estudiante

Sin dudas considero que lo que he realizado aquí no es el máximo que puedo, el tema de las validaciones y pudo haber centrado más en estos últimos días, por lo menos para tener esas secciones completas, si lo volviera a hacer, empezaría por crear todo el sistema y luego trabajar a fondo las validaciones.

Las funciones auxiliares fueron un buen punto de vista nuevo para este proyecto, muchas veces pensé en anidar todo el código de por ejemplo, identificar si un elemento se repite, entonces al darme cuenta que tendría que copiar eso varias veces, mejor lo convertí en una función auxiliar, esto si hubiese sido más responsable, me hubiese ayudado a ganar bastante más tiempo.

En este caso el manejo de archivos al principio fue un caos, en algunas ocasiones el usar **with open (archivo, "…") as archivo:** ,e resultaba confuso, pues tenía dos parámetros ahí llamados exactamente igual, la próxima vez buscaría una mejor forma de manejar este problema, al igual que los while anidados (*esta fue parte de la lógica en la que me tuve que apoyar con IA para entender de mejor manera*), siento que se ve ordenada, pero a la hora de leerla me tomaba un tiempo cada vez que abría el archivo al día siguiente para volver a concentrarme en el flujo.

Me confié con la simplicidad del proyecto, si bien el tiempo no es excusa pues había bastante, fue un grave error suponer que como él proyecto fue parecido al primero del semestre pasado, este seguiría un mismo camino, lo deje para lo último y este resultado me ha dejado insatisfecho,